👩‍⚕️ Assignment: Healthcare Management REST API

Technology Stack: ASP.NET Core Web API + Entity Framework Core (Code First) + SQL Server

---

🎯 Objective

Design and implement a **Healthcare Management System API** that manages patients, doctors, appointments, and prescriptions using **Code First approach in Entity Framework Core**.

📌 Functional Requirements

1. Patient Management

- Create a Patient
- Get all Patients
- Get Patient by Id
- Update Patient details
- Delete Patient

Patient Fields:

- PatientId (PK)
- Name
- Age
- Gender
- ContactNumber
- Address
- CreatedDate

---

2. Doctor Management

- Add Doctor

- Get all Doctors
- Get Doctor by specialization

Doctor Fields:

- DoctorId (PK)
- Name
- Specialization
- Experience (Years)
- ConsultationFee

---

3. Appointment Management

- Book Appointment
- Get all Appointments
- Get Appointments by PatientId
- Cancel Appointment

Appointment Fields:

- AppointmentId (PK)
- PatientId (FK)
- DoctorId (FK)
- AppointmentDate
- Status (Booked / Cancelled / Completed)

---

4. Prescription Management

- Add Prescription for Appointment
- Get Prescription by AppointmentId

Prescription Fields:

- PrescriptionId (PK)

- AppointmentId (FK)
- Diagnosis
- Medicines
- Notes

🧱 Technical Requirements

1. Entity Framework Core (Code First)

- Create DbContext class
- Configure relationships using:
  - One-to-Many (Patient → Appointments)
  - One-to-Many (Doctor → Appointments)
  - One-to-One (Appointment → Prescription)
- Apply **Fluent API / Data Annotations**

---

2. Migrations

- Create Initial Migration
- Update Database using:

```
Add-Migration InitialCreate

Update-Database
```

---

3. Layered Architecture (Mandatory)

Organize project into:

- **Models (Entities)**
- **Data Layer (DbContext)**
- **Repository Layer**
- **Service Layer**

- **Controller Layer**

---

4. API Design (RESTful)

Use proper HTTP methods:

Operation	Method	Endpoint
Create Patient	POST	/api/patients
Get All Patients	GET	/api/patients
Get by Id	GET	/api/patients/{id}
Update	PUT	/api/patients/{id}
Delete	DELETE	/api/patients/{id}

👉 Apply same pattern for Doctors, Appointments, Prescriptions

---

5. Validation

- Use Data Annotations:
  - [Required]
  - [MaxLength]
  - [Range]
- Return proper validation errors

---

6. Exception Handling

- Implement global exception handling middleware
- Return standardized response:

```
{
  "message": "Error occurred",
  "details": "..."
```

}

---

7. Dependency Injection

- Register:
  - DbContext
  - Repositories
  - Services

---

8. Swagger Integration

- Enable Swagger UI
- Test all endpoints

📂 **Expected Project Structure**

```
HealthcareAPI/
│
├── Controllers/
├── Models/
├── Data/
├── Repositories/
├── Services/
├── DTOs/
└── Program.cs
```

🧪 **Sample API Scenario**

1. Create Patient
2. Create Doctor
3. Book Appointment
4. Add Prescription
5. Fetch Patient Appointments

💡 **Tips (for Students)**

- Start with **Entities → DbContext → Migration → Controllers**
- Test each module independently
- Use **Postman or Swagger** frequently
- Keep code modular and clean